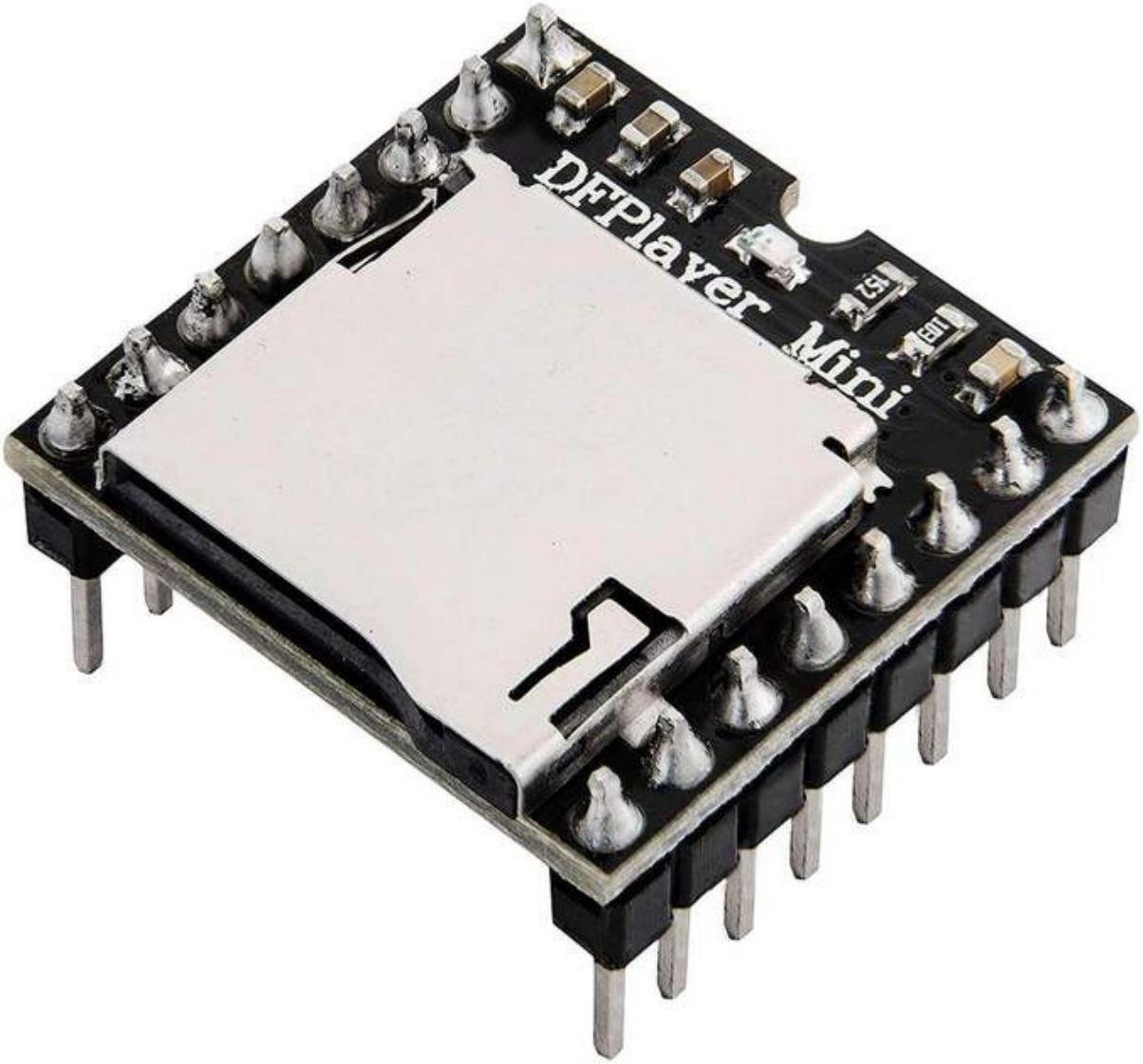
Bienvenue !

Nous vous remercions d'avoir acheté notre *mini-module AZ-Delivery MP3 DFPlayer*. Dans les pages suivantes, nous allons vous expliquer comment utiliser et configurer cet appareil pratique.

Amusez-vous bien !



Zones d‘application

Éducation et enseignement : Utilisation dans les écoles, universités et établissements de formation pour enseigner les bases de l‘électronique, de la programmation et des systèmes embarqués. Recherche et développement : Utilisation dans des projets de recherche et développement pour créer des prototypes et des expériences dans les domaines de l‘électronique et de l‘informatique. Développement de prototypes : utilisation dans le développement et le test de nouveaux circuits et dispositifs électroniques. Projets de loisirs et de création : utilisé par les passionnés d‘électronique et les amateurs pour développer et mettre en œuvre des projets de bricolage.

Connaissances et compétences requises

Compréhension de base de l‘électronique et du génie électrique. Connaissance de la programmation, notamment du langage de programmation C/C++. Capacité à lire des schémas et à concevoir des circuits simples. Expérience de travail avec des composants électroniques et de la soudure.

Des conditions de fonctionnement

Le produit ne peut être utilisé qu‘avec les tensions spécifiées dans la fiche technique pour éviter tout dommage. Une source d’alimentation CC stabilisée est requise pour le fonctionnement. Lors de la connexion à d‘autres composants et circuits électroniques, les limites maximales de courant et de tension doivent être respectées pour éviter les surcharges et les dommages.

Conditions environnementales

Le produit doit être utilisé dans un environnement propre et sec pour éviter les dommages causés par l‘humidité ou la poussière. Protéger le produit des rayons directs du soleil (UV)

Utilisation prévue

Le produit est conçu pour être utilisé dans des environnements d’enseignement, de recherche et de développement. Il est utilisé pour développer, programmer et prototyper des projets et des applications électroniques. Le produit Sensor n‘est pas conçu comme un produit de consommation fini, mais plutôt comme un outil destiné aux utilisateurs techniquement avertis, notamment les ingénieurs, les développeurs, les chercheurs et les étudiants.

Utilisation inappropriée et prévisible

Le produit n‘est pas adapté à un usage industriel ou à des applications liées à la sécurité. L‘utilisation du produit dans des dispositifs médicaux ou à des fins de voyages aériens et spatiaux n‘est pas autorisée.

élimination

Ne pas jeter avec les ordures ménagères ! Votre produit est conforme à celui européen Directive relative aux déchets d‘équipements électriques et électroniques devant être éliminés de manière respectueuse de l‘environnement. Les matières premières précieuses qu‘elles contiennent peuvent être recyclées devenir. L‘application de cette directive contribue à la protection de l‘environnement et de la santé. Utilisez le point de collecte mis en place par votre commune pour rapporter et Recyclage des anciens appareils électriques et électroniques. N° d‘enregistrement DEEE : DE 62624346

décharge électrostatique

Attention : Les décharges électrostatiques peuvent endommager le produit. Remarque : Mettez-vous à la terre avant de toucher le produit, par exemple en portant un bracelet antistatique ou en touchant une surface métallique mise à la terre.

consignes de sécurité

Bien que notre produit soit conforme aux exigences de la directive RoHS (2011/65/UE) et ne contienne aucune substance dangereuse en quantités supérieures aux limites autorisées, des résidus peuvent toujours être présents. Respectez les consignes de sécurité suivantes pour éviter les risques chimiques : Attention : La soudure peut produire des fumées pouvant être nocives pour la santé. Remarque : utilisez un extracteur de fumées de soudure ou travaillez dans un endroit bien ventilé. Si nécessaire, portez un masque respiratoire. Attention : Certaines personnes peuvent être sensibles à certains matériaux ou produits chimiques contenus dans le produit. Remarque : En cas d‘irritation cutanée ou de réactions allergiques, cesser l‘utilisation et, si nécessaire, consulter un médecin. Attention : Gardez le produit hors de portée des enfants et des animaux domestiques pour éviter tout contact accidentel et l‘ingestion de petites pièces. Remarque : Conservez le produit dans un récipient sûr et fermé lorsqu‘il n‘est pas utilisé. Attention : Évitez tout contact du produit avec des aliments et des boissons. Remarque : Ne stockez pas et n‘utilisez pas le produit à proximité d‘aliments pour éviter toute contamination. Bien que notre

produit soit conforme aux exigences de la directive RoHS (2011/65/UE) et ne contienne aucune substance dangereuse en quantités supérieures aux limites autorisées, des résidus peuvent toujours être présents. Respectez les consignes de sécurité suivantes pour éviter les risques chimiques : Attention : La soudure peut produire des fumées pouvant être nocives pour la santé. Remarque : utilisez un extracteur de fumées de soudure ou travaillez dans un endroit bien ventilé. Si nécessaire, portez un masque respiratoire. Attention : Certaines personnes peuvent être sensibles à certains matériaux ou produits chimiques contenus dans le produit. Remarque : En cas d'irritation cutanée ou de réactions allergiques, cesser l'utilisation et, si nécessaire, consulter un médecin. Attention : Gardez le produit hors de portée des enfants et des animaux domestiques pour éviter tout contact accidentel et l'ingestion de petites pièces. Remarque : Conservez le produit dans un récipient sûr et fermé lorsqu'il n'est pas utilisé. Attention : Évitez tout contact du produit avec des aliments et des boissons. Remarque : Ne stockez pas et n'utilisez pas le produit à proximité d'aliments pour éviter toute contamination. Le produit contient des composants électroniques sensibles et des arêtes vives. Une manipulation ou un assemblage incorrect peut entraîner des blessures ou des dommages. Respectez les consignes de sécurité suivantes pour éviter les risques mécaniques : Attention : le circuit imprimé et les connecteurs du produit peuvent présenter des arêtes vives. Soyez prudent pour éviter les coupures. Remarque : Portez des gants de protection appropriés lors de la manipulation et de l'assemblage du produit. Attention : évitez toute pression excessive ou toute contrainte mécanique sur la carte et les composants. Remarque : montez le produit uniquement sur des surfaces stables et planes. Utilisez des entretoises et des boîtiers appropriés pour minimiser les contraintes mécaniques. Attention : assurez-vous que le produit est solidement fixé pour éviter tout glissement ou chute accidentelle. Remarque : Utilisez un support approprié ou un montage sécurisé dans des boîtiers ou sur des plaques de montage. Attention : assurez-vous que toutes les connexions des câbles sont connectées de manière sécurisée et correcte pour éviter les tensions et les débranchements accidentels. Remarque : Acheminez les câbles de manière à ce qu'ils ne soient pas sous tension et ne présentent pas de risque de trébuchement. Le produit fonctionne avec des tensions et des courants électriques qui, s'ils sont mal utilisés, peuvent entraîner des chocs électriques, des courts-circuits ou d'autres dangers. Respectez les consignes de sécurité suivantes pour éviter les risques électriques : Attention : utilisez le produit uniquement avec les tensions spécifiées. Remarque : Les limites de performances du produit se trouvent dans la fiche technique associée Attention : Évitez les courts-circuits entre les connecteurs et les composants du produit Remarque : Assurez-vous qu'aucun objet conducteur ne touche ou ne ponte le circuit imprimé. Utilisez des outils isolés et faites attention à la disposition des connexions. Attention : N'effectuez aucune intervention sur le produit lorsqu'il est connecté à une source d'alimentation. Remarque : Débranchez le produit de l'alimentation avant d'effectuer des modifications de circuit ou de connecter ou de retirer des composants. Attention : Ne dépassez pas les courants nominaux spécifiés pour les entrées et sorties du produit. Remarque : Les limites de performance du produit se trouvent dans les spécifications techniques ou dans la fiche technique Attention : Assurez-vous que les sources d'alimentation utilisées sont stables et correctement dimensionnées. Remarque : utilisez uniquement des alimentations testées et adaptées pour éviter les fluctuations de tension et les surcharges. Attention : Maintenez une distance suffisante par rapport aux pièces sous tension pour éviter tout contact accidentel. Remarque : Assurez-vous que le câblage est disposé de manière sûre et claire en fonction de la tension utilisée. Attention : Utilisez des boîtiers isolants ou des capots de protection pour protéger le produit du contact direct. Remarque : placez le produit dans un boîtier non conducteur pour éviter tout contact accidentel et court-circuit. Le produit et les composants qu'il contient peuvent devenir chauds pendant le fonctionnement. Une mauvaise manipulation ou une surcharge du produit peut entraîner des brûlures, des dommages ou un incendie. Respectez les consignes de sécurité suivantes pour éviter les risques thermiques : Attention : assurez-vous que le produit est utilisé dans les températures de fonctionnement recommandées. Remarque : La plage de températures de fonctionnement recommandée est généralement comprise entre-40 °C et +85 °C. Vérifiez les informations spécifiques dans la fiche technique du produit. Attention : Ne placez pas le produit à proximité de sources de chaleur externes telles que des radiateurs ou la lumière directe du soleil. Remarque : Assurez-vous que le produit est utilisé dans un endroit frais et bien ventilé. Attention : Assurez-vous que le produit est bien ventilé pour éviter toute surchauffe. Remarque : utilisez des ventilateurs ou des dissipateurs de chaleur lorsque vous utilisez le produit dans une enceinte fermée ou dans un environnement avec une circulation d'air limitée. Attention : Montez le produit sur des surfaces résistantes à la chaleur et dans des boîtiers résistants à la chaleur. Remarque : Utilisez des matériaux de boîtier pouvant résister à des températures élevées pour éviter tout dommage ou tout risque d'incendie. Attention : Mettez en place une surveillance de la température lors de l'utilisation d'un boîtier et, si nécessaire, des mécanismes de protection qui arrêtent le produit en cas de surchauffe. Remarque : Remarque : Utilisez des capteurs de température et un logiciel approprié pour surveiller la température du produit et arrêter le système si nécessaire. Attention : évitez les surcharges pouvant provoquer un échauffement excessif des composants. Remarque : Pour éviter toute surchauffe, ne dépassez pas les limites de courant et de tension spécifiées. Attention : Les courts-circuits peuvent générer une chaleur importante et provoquer des incendies. Remarque : Assurez-vous que toutes les connexions sont correctes et sécurisées et qu'aucun objet conducteur ne peut provoquer accidentellement des courts-circuits.

Le mini module MP3 DFPlayer est un module MP3 petit et abordable avec une sortie directe vers le haut-parleur ou les écouteurs. Le module peut être utilisé comme module autonome avec une batterie, un haut-parleur et des boutons poussoirs ou en combinaison avec une carte Atmega328p ou toute autre carte avec des capacités USART.

Le module prend en charge les formats audio courants tels que *MP3*, *WAV* et *WMA*. Il prend également en charge les cartes TF avec les systèmes de fichiers *FAT16* et *FAT32.* Vous pouvez jouer de la musique à travers un simple port série sans aucune opération complexe.

Le module est livré avec deux connecteurs mâles à *8* broches pré-soudés, et il dispose d'un emplacement pour carte microSD. Il y a également une LED rouge à bord, utilisée pour indiquer l'état de lecture des chansons тxe. La DEL est connectée à la broche *BUSY* du module. L'état *allumé* de la LED indique que la chanson est en cours de lecture.

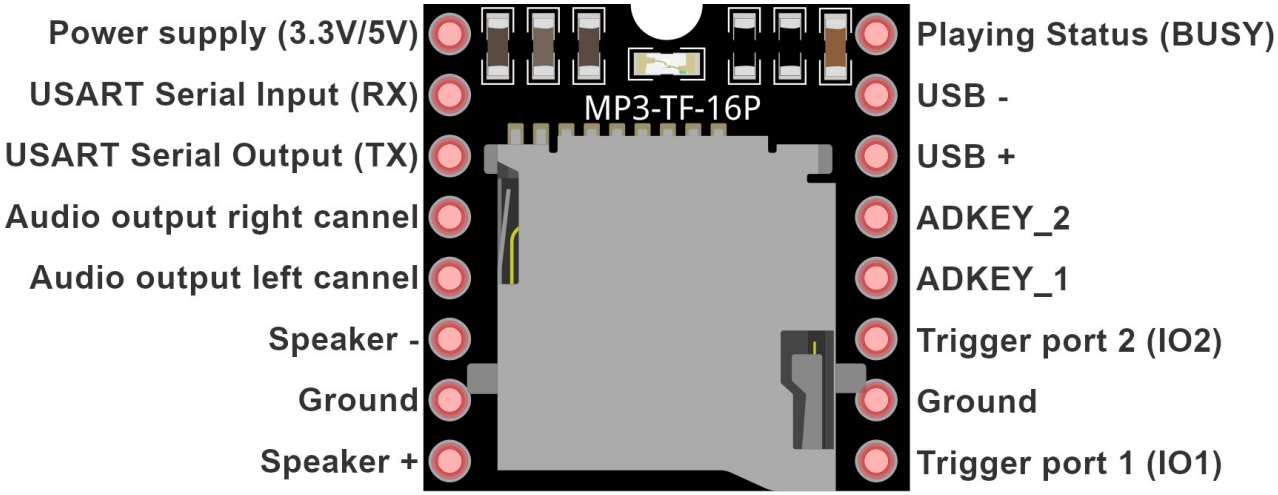
Spécification :

"	Plage de tension de fonctionnement :	de 3,2V à 5V DC
"	Courant de veille :	20mA
"	Température de fonctionnement :	de -40℃ à 70℃
"	Port UART :	Série standard (niveau TTL)
"	Vitesse de transmission :	Réglable (par défaut 9.600)
"	Equalizer :	6 niveaux, réglables
"	Niveaux de volume :	30 niveaux, réglables
"	Taux d'échantillonnage (kHz) :	8/11.025/12/16/22/24/32/44.1/48
"	Sortie :	- Plage DAC 24 bits 90dB, - Support SNR 85dB
"	Puissance de sortie :	3W
"	Résistance de l'enceinte :	3Ω (maximum 4Ω)
"	Système de fichiers :	FAT16 ou FAT32
"	Soutien maximal :	- 32GB de la carte TF, - Disque flash USB de 32 Go, - 64 Mo d'octets NORFLASH
"	Modes de contrôle :	- Mode de contrôle des E/S, - Mode série, - Mode de contrôle du bouton AD

"Fonction d'attente du son publicitaire (la musique peut être suspendue ; lorsque la publicité est terminée, la musique continue d'être diffusée)

"Les données audio sont classées par dossiers. Le module prend en charge jusqu'à 100 dossiers et chaque dossier peut contenir jusqu'à 255 chansons.

Le brochage



Les broches USART sont utilisées pour la communication en série. Si vous constatez un bruit élevé, connectez une résistance de *1kΩ* à la broche TX, en série.

Les broches des canaux de sortie audio sont utilisées commebroches DAC (Digital to Analog Converter), et vous devez les connecter à l'amplificateur externe.

Les broches des haut-parleurs sont des broches de l'amplificateur intégré de *3W,* et vous pouvez les connecter directement au haut-parleur externe (*8Ω* max).

Les broches ADKEY sont utilisées pour le mode de contrôle AD.

Les broches du port de déclenchement sont utilisées pour régler les niveaux de volume ou pour changer de morceau (matériel). Ces broches sont actives en *BAS*.

Les broches USB sont utilisées pour la connexion à la clé USB.

Alimentation et broche BUSY

Le module supporte une tension de fonctionnement de *3,2V* à *5V DC*. Connecter l'alimentation externe entre la broche d'alimentation et la broche de mise à la terre.

La tension du niveau logique *TTL du* port série du module est de *3,3V*, donc si vous utilisez des niveaux de *5V* (par exemple la carte Atmega328p), connectez en série une résistance de plus de *1kΩ* à la broche RX du module. Si vous n'utilisez pas cette résistance, vous constaterez un bruit élevé sur les canaux de sortie audio (haut-parleurs).

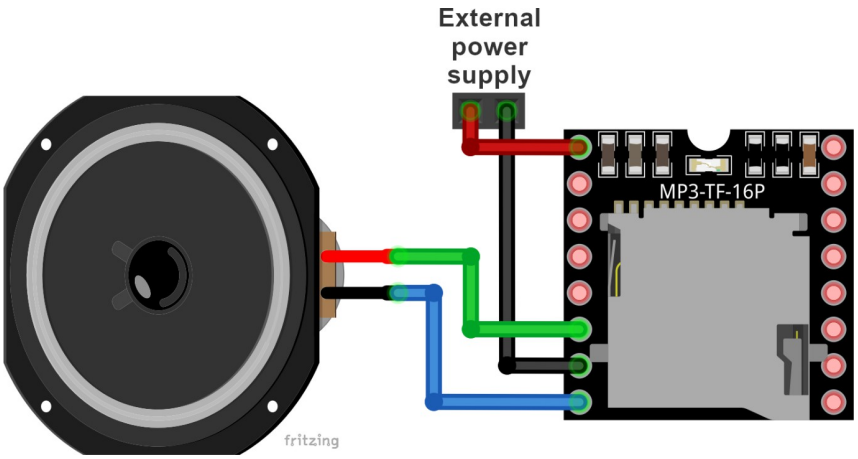
La broche d'état de lecture ou broche *BUSY* est utilisée pour indiquer l'état de lecture d'un morceau. *L'*état *BAS* de cette broche indique que le module est en train de jouer un morceau, et l'état *HAUT* indique que le module ne joue aucun morceau.

Canaux de sortie audio et broches de haut-parleur

La puce principale du module est un convertisseur numérique-analogique de *24* bits (DAC en abrégé). La puce possède deux broches DAC qui sont connectées directement aux canaux de sortie audio du module. Vous devez connecter un amplificateur externe sur les broches des canaux de sortie audio afin d'utiliser les capacités DAC du module. Si vous souhaitez utiliser ces broches, activez d'abord la sortie DAC en envoyant la commande correspondante à la puce (plus loin dans le texte).

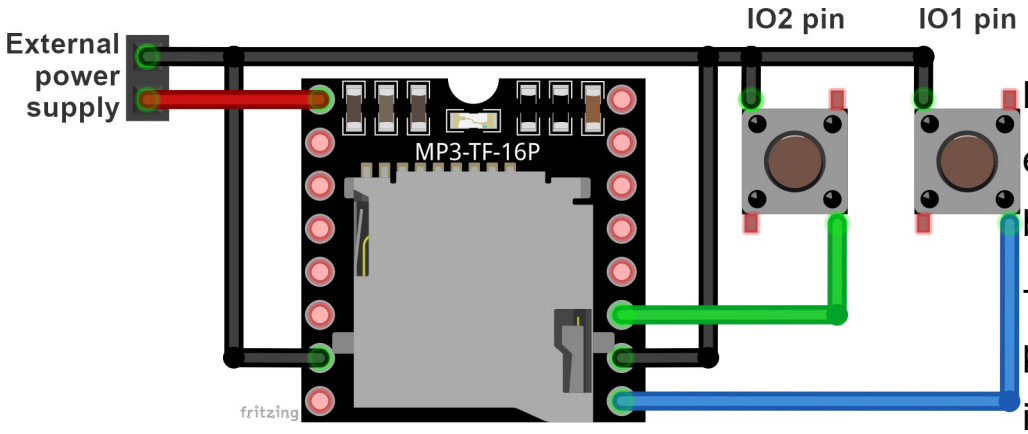
Le module est équipé d'un circuit d'amplification de *3 W. Le cœur de ce circuit est un dispositif appelé "amplificateur audio 8002". Le* cœur de ce circuit est un dispositif appelé "*amplificateur audio 8002*", un circuit intégré (CI) à *8* broches dont la sortie est connectée aux broches `Haut-parleur+` et `Haut-parleur-. La` sortie du circuit intégré *8002* est un son mono.

Connectez le haut-parleur externe au module comme indiqué sur le schéma de connexion ci-dessous :



Ports de déclenchement (broches IO)

Ces broches sont utilisées pour la commutation des morceaux et le réglage des niveaux de volume via le matériel. Connectez les boutons poussoirs à ces broches comme indiqué sur le schéma de connexion :



Lorsque le bouton-poussoir est en position état de repos, les broches diagonales du bouton-poussoir ne sont pas connectées. Lorsque vous appuyez sur le bouton-poussoir, les broches diagonales du bouton-poussoir sont connectées, ce qui met le bouton-poussoir dans un état actif.

Une pression courte sur le bouton-poussoir *IO2* permet de jouer la chanson suivante. Une pression longue sur le bouton *IO2*

permet d'augmenter le volume.

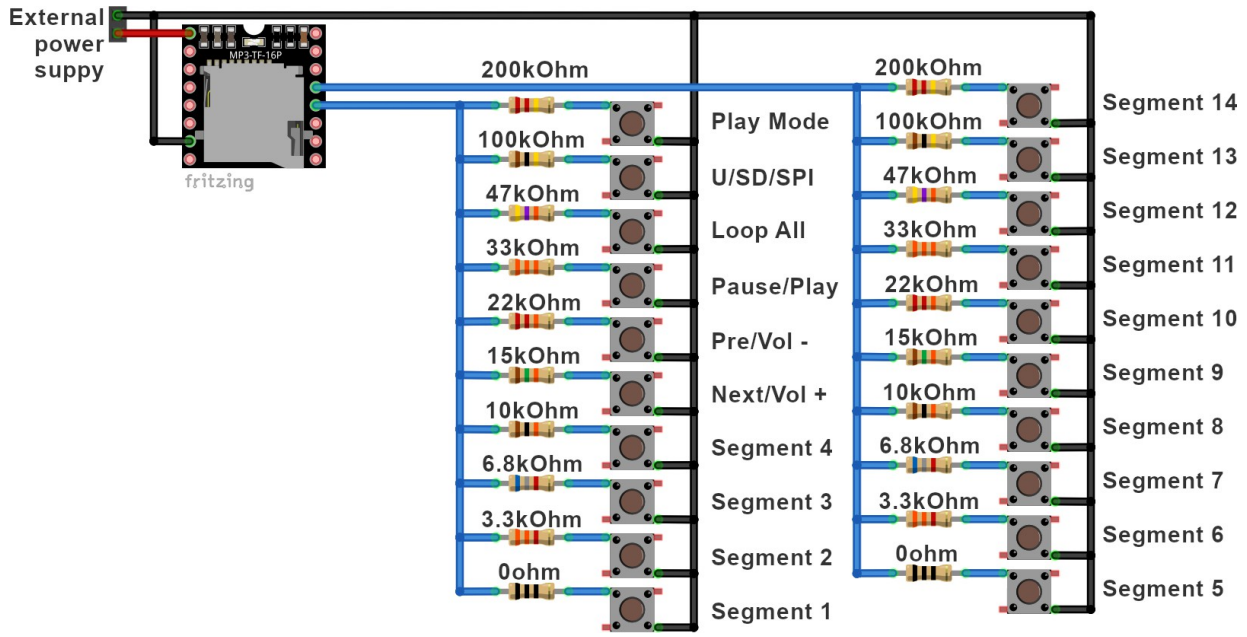
Une pression courte sur le bouton-poussoir *IO1* permet de jouer la chanson précédente. Une pression longue sur le bouton-poussoir

Le bouton-poussoir *IO2* diminue le niveau du volume.

La durée de la poussée longue est supérieure à une seconde. Tout ce qui est inférieur à une seconde est considéré comme une poussée courte.

Broches ADKEY

La fonctionnalité AD de la puce embarquée dans le module vous permet de connecter *20* résistances avec des boutons sur deux ports AD du module, comme le montre le schéma de connexion ci-dessous :



REMARQUE : Il est nécessaire que l'alimentation électrique soit aussi stable que possible pour que le système fonctionne correctement !

Une brève pression sur le bouton *Play Mode* permet d'*interrompre* ou de *ne pas interrompre la* lecture. Cela signifie que la lecture sera ou ne sera pas interrompue par une publicité. Il n'y a pas de fonction d'appui long pour ce bouton.

Une pression brève sur le bouton *U/TF/SPI* fait basculer le dispositif de lecture sur l'un des éléments suivants : *U* = disque flash USB, *TF* = carte SD, *SPI* = NORFLASH ou *Veille*. Il n'y a pas de fonction d'appui long pour ce bouton.

Une brève pression sur le bouton `Loop All` permet d'activer le mode de lecture en *boucle* ou de *le désactiver.*

pas de mise en boucle de toutes les chansons. Il n'y a pas de fonction d'appui long pour ce bouton.

Une brève pression sur le bouton `Pause/Play` met en pause ou joue le morceau sélectionné. Il n'y a pas de fonction d'appui long pour ce bouton.

Une pression courte sur le bouton `Pre/Vol+` permet de jouer la chanson précédente. Une pression longue sur le *bouton* `Pre/Vol+` augmente le niveau du volume.

Une pression brève sur le bouton `Next/Vol-` permet de passer à la chanson suivante. Une pression longue sur le *bouton* `Pre/Vol+` diminue le niveau du volume.

Une pression courte sur le bouton `Segment1` joue la chanson numéro 1. Une pression longue sur le *bouton* `Segment1` répète la lecture du même morceau.

Les fonctions sont les mêmes pour tous les autres boutons de `segment`, à l'exception du numéro de morceau qui diffère.

Port série

Les broches *RX* et *TX* sont utilisées pour établir une communication série avec le microcontrôleur externe. N'oubliez pas de connecter une résistance à la broche *RX* lorsque vous utilisez une logique *TTL 5V.* Le port série du module prend en charge le mode de communication série asynchrone. Le débit en bauds par défaut de la communication série est de *9600bps* et il est réglable par logiciel.

Vous pouvez utiliser le port série pour envoyer des commandes simples au module et contrôler ainsi de nombreuses fonctions prises en charge par le module. Pour en savoir plus sur les commandes, voir le chapitre suivant.

Spécifications

Vitesse de transmission par défaut :	9600bps
Bits de données :	1
Sortie de caisse :	aucun
Contrôle de flux :	aucun

Format de la commande

Pour envoyer une commande au module, suivez le format spécifique :

$SB VB LB CMD ACK DATA1 DATA2 CHKS1 CHKS2 $EB

Marque	Octet	Description des octets
$SB	*0x7E*	Octet de départ
VB	*0xFF*	Octet de version
LB	*0xxx*	Le nombre d'octets de la commande sans les octets de début et de fin (dans notre cas, *0x06*)
CMD	*0xxx*	Tels que *PLAY* et *PAUSE*, etc.
ACK	*0xxx*	Octet d'acquittement *0x00* = pas d'acquittement, *0x01* = acquittement
DATA1	*0xxx*	Octet de données de poids fort
DATA2	*0xxx*	Octet de données basses
CHKS1	*0xxx*	Octet de poids fort de la somme de contrôle
CHKS2	*0xxx*	Octet de poids faible de la somme de contrôle
$EB	*0xEF*	Octet de fin

L'octet d'accusé de réception est utilisé pour obtenir des données du module. S'il est réglé sur *0x00,* aucune donnée ne sera envoyée par le module et s'il est réglé sur *0x01,* vous obtiendrez des données de réponse du module. La longueur des données n'est pas limitée, mais elle est généralement de deux *octets*(*data1* et *data2*).

Pour envoyer une commande spécifique, il suffit d'envoyer un octet par octet en série via l'interface série du logiciel (vous verrez cela plus loin dans le code).

Structure des dossiers et noms des chansons

Le module prend en charge plusieurs types de dossiers et de noms spécifiques pour les chansons. Les noms des dossiers sont des nombres, à l'exception des dossiers "*mp3*" et "*ADVERT*". Les noms des chansons doivent commencer par un chiffre, suivi d'une chaîne de caractères sans espace. Exemple de nom de chanson : `0001_Linking_Park_In_The_End.mp3`

Il existe *5* types de dossiers.

Le premier est un type de dossier qui peut contenir *256* chansons. Le module prend en charge *256* de ces dossiers au total. Les noms de ces dossiers sont des nombres compris entre *000* et *255*. Les noms des chansons dans ces dossiers commencent par des nombres compris entre *000* et *255*. Le module ne prend pas en charge les sous-dossiers dans ces dossiers.

Le second est un type de dossier qui peut contenir *3000* chansons. Le module prend en charge *16* de ces dossiers au total. Les noms de ces dossiers sont des nombres compris entre *00* et *15*. Les noms des chansons dans ces dossiers commencent par des nombres compris entre *0000* et *2999*. Le module ne prend pas en charge les sous-dossiers de ces dossiers.

Le troisième est un dossier appelé "*mp3*", qui peut lui aussi contenir *3000* chansons. Le module prend en charge un dossier "*mp3*" au total. Les noms des chansons dans ce dossier commencent par des nombres compris entre *0000* et *2999*. Le module ne prend pas en charge les sous-dossiers de ce

dossier.

Le quatrième dossier, appelé "*ADVERT*", peut lui aussi contenir *3000* chansons et est utilisé pour les chansons publicitaires. Le module prend en charge un dossier "*ADVERT*" au total. Les noms des chansons dans ce dossier commencent par des nombres compris entre *0000* et *2999*. Le module ne prend pas en charge les sous-dossiers de ce dossier.

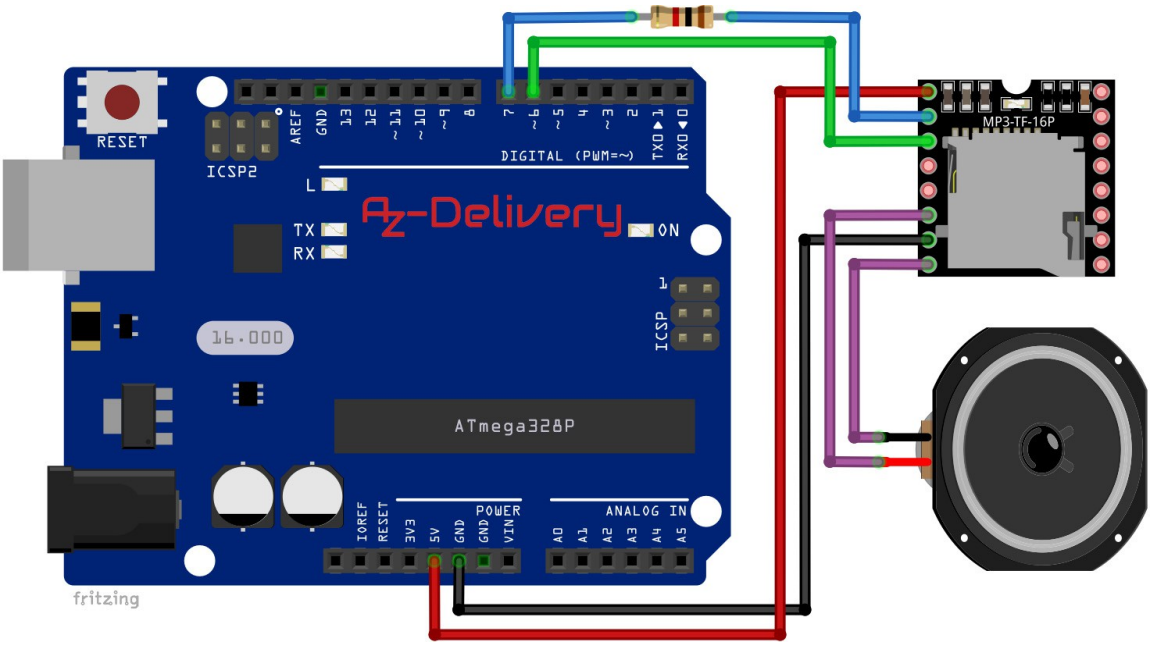
Le cinquième type de dossier est le dossier "*racine*". S'il s'agit du seul dossier de la carte SD ou de la mémoire USB (aucun autre dossier), ce dossier peut contenir jusqu'à *65536* chansons. Ce dossier peut avoir des sous-dossiers, y compris n'importe quel type de dossier ou tous les types de dossier. Le dossier "*racine*" peut contenir à la fois des chansons et des sous-dossiers.

Exemple de structure de dossier :

```
racine
--- 0001r.mp3
--- 0002r.mp3
--- 0003r.mp3
--- 0004r.mp3
--- 0001
-----------0001x.mp3
--- --- 0002x.mp3
--- 0002
--- --- 0001y.mp3
-----mp3
--- --- 0001m.mp3
--- --- 0002m.mp3
--- --- 0003m.mp3
--- ADVERT
--- --- 0001a.mp3
--- --- 0002a.mp3
```

Envoi de commandes au module

Afin d'envoyer des commandes au module, connectez le module à l'Atmega328p comme indiqué sur le schéma de connexion ci-dessous :



Broche du module	**>**	**Mc pin**	
VCC	>	5V	**Fil rouge**
RX	>	D7 (via une résistance de 1kΩ)	**Fil bleu**
TX	>	D6	**Fil vert**
GND	>	GND	**Fil noir**

Broche du module	**>**	**Broche du haut-parleur**	
SPK -	>	Un côté de l'enceinte	**Fil violet**
SPK+	>	L'autre côté de l'enceinte	**Fil violet**

NOTE : Vous pouvez utiliser n'importe quelle autre carte avec un microcontrôleur qui a des capacités USART.

Nous utilisons l'interface série créée dans le logiciel sur les broches E/S numériques *6* et *7* de l'Atmega328p, car l'Atmega328p utilise les broches série matérielles (broches E/S numériques *0* et *1*) pour programmer le microcontrôleur principal.

Le microcontrôleur ne peut pas envoyer de commandes pour contrôler le module tant que l'initialisation du module n'est pas terminée et que les données ne sont pas renvoyées. Sinon, les commandes envoyées par le microcontrôleur seront ignorées et cela affectera également le processus d'initialisation.

Sauf indication contraire (par l'envoi d'une commande après l'initialisation), lorsque le module est *mis sous tension*, il lit d'abord la carte SD et, si la carte SD n'est pas disponible, il passe au disque flash USB.

Exemple de croquis :

```
#include "SoftwareSerial.h"
#define Start_Byte 0x7E
#define Version_Byte 0xFF
#define Command_Length 0x06
#define End_Byte 0xEF
// Returns info with command 0x41 [0x01: info, 0x00: no
info]
#define Acknowledge 0x01
SoftwareSerial mySerial(6, 7); // RX, TX
byte receive_buffer[10] = {0, 0, 0, 0, 0, 0, 0, 0, 0, 0};
char data; // Used for received commands from Serial Monitor
byte volume = 0x00; // Used to store current volume level
bool mute_state = false; // Used to toggle mute state
// Excecute the command and parameters
void execute_CMD(byte Command, byte Data1, byte Data2) {
 // Calculate the checksum (2 bytes)
 word Checksum = -( Version_Byte + Command_Length + Command
+
Acknowledge + Data1 + Data2);
 // Build the command
 byte command_line[10] = { Start_Byte, Version_Byte,
 Command_Length, Command, Acknowledge,
 Data1, Data2, highByte(Checksum),
 lowByte(Checksum), End_Byte};
 // Send the command line to the module
 for(byte k = 0; k < 10; k++) {
 mySerial.write(command_line[k]);
 }
}
```

```
void reset_rec_buf() {
 for(uint8_t i = 0; i < 10; i++) {
 receive_buffer[i] = 0;
 }
}
bool receive() {
 reset_rec_buf();
 if(mySerial.available() < 10) {
 return false;
 }
 for(uint8_t i = 0; i < 10; i++) {
 short b = mySerial.read();
 if(b == -1) {
 return false;
 }
 receive_buffer[i] = b;
 }
 // When you reset the module in software,
 // received buffer elements are shifted.
 // To correct that we do the following:
 short b = receive_buffer[0];
 for(uint8_t i = 0; i < 10; i++) {
 if(i == 9) {
 receive_buffer[i] = b;
 }
 else {
 receive_buffer[i] = receive_buffer[i+1];
 }
 } // End correcting receive_buffer
 return true;
}
```

```
void print_received(bool print_it) {
 if(print_it) {
 if(receive()) {
 for(uint8_t i = 0; i < 10; i++) {
 Serial.print(receive_buffer[i], HEX); Serial.print("\t");
 }
 Serial.println();
 }
 }
 else { receive(); }
}
void module_init() {
 execute_CMD(0x0C, 0, 0); delay(1000); // Reset the module
 print_received(false); delay(100);
 Serial.print("SDON\t");
 print_received(true); delay(100);
 playFirst();
 setVolume(0x09);
}
void play_first() {
 Serial.print("PLYFST\t");
 execute_CMD(0x03, 0, 1); delay(100); // Play first song
 print_received(false); delay(100);
 execute_CMD(0x45, 0, 0); delay(100); // Get playback status
 print_received(false); delay(100);
 print_received(true); delay(100);
}
```

```
void set_volume(uint8_t volume) {
 Serial.print("SETVOL\t");
 execute_CMD(0x06, 0, volume); delay(100); // Set volume level
 print_received(false); delay(100);
 execute_CMD(0x43, 0, 0); delay(100); // Get volume level
 print_received(false); delay(100);
 print_received(true); delay(100);
}
void play_next() {
 Serial.print("NEXT\t");
 execute_CMD(0x01, 0, 0); delay(100);
 print_received(false); delay(100);
 execute_CMD(0x4C, 0, 0); delay(100); // Get current song played
 print_received(false); delay(100);
 print_received(true); delay(100);
}
void mute() {
 mute_state = !mute_state;
 if(mute_state) {
 execute_CMD(0x43, 0, 0); delay(100); // Return volume leve
 print_received(false); delay(100);
 print_received(false); delay(100);
 volume = receive_buffer[6];
 Serial.print("MUTE\t");
 execute_CMD(0x06, 0, 0x00); delay(100); // Set volume level
 print_received(false); delay(100);
 execute_CMD(0x43, 0, 0); delay(100); // Get volume level
 print_received(false); delay(100);
 print_received(true); delay(100);
 }
```

```
// one tab
 else {
 Serial.print("VOL\t");
 execute_CMD(0x06, 0, volume); delay(100); // Set previous vol
 print_received(false); delay(100);
 execute_CMD(0x43, 0, 0); delay(100); // Get volume level
 print_received(false); delay(100);
 print_received(true); delay(100);
 }
}
void random_play() {
 // Random plays all songs, loops all, repeats songs in playback
 execute_CMD(0x18, 0, 0);
 delay(100);
 Serial.print("RNDM\t");
 print_received(false);
 delay(100);
 execute_CMD(0x4C, 0, 0); // Get current song played
 delay(100);
 print_received(false);
 delay(100);
 print_received(true);
 delay(100);
}
```

```
void query_status() {
 execute_CMD(0x42, 0, 0); delay(100); // Get status of module
 print_received(false); delay(100);
 Serial.print("STATUS\t");
 print_received(true); delay(100);
 execute_CMD(0x43, 0, 0); delay(100); // Get volume level
 print_received(false); delay(100);
 Serial.print("VOLUME\t");
 print_received(true); delay(100);
 execute_CMD(0x44, 0, 0); delay(100); // Get EQ status
 print_received(false); delay(100);
 Serial.print("EQ\t");
 print_received(true); delay(100);
 execute_CMD(0x45, 0, 0); delay(100); // Get playback status
 print_received(false); delay(100);
 Serial.print("PLYBCK\t");
 print_received(true); delay(100);
 execute_CMD(0x46, 0, 0); delay(100); // Get software version
 print_received(false); delay(100);
 Serial.print("SFVER\t");
 print_received(true); delay(100);
 // Get total number of files on storage device
 execute_CMD(0x48, 0, 0); delay(100);
 print_received(false); delay(100);
 Serial.print("FILES\t");
 print_received(true); delay(100);
 execute_CMD(0x4C, 0, 0); delay(100); // Get current song played
 print_received(false); delay(100);
 Serial.print("CRRTRK\t");
 print_received(true); delay(100);
}
```

```
void setup() {
 Serial.begin(115200);
 mySerial.begin(9600); delay(1000);
 Serial.println("\nInitialization");
 module_init();
}
void loop() {
 print_received(true);

 while(Serial.available() > 0) {
 data = Serial.read();
 // Serial.println(data, HEX); // For debugging
 if(data != "/n") {
 if(data == 'N') {
 Serial.println("\nPlay next song");
 play_next();
 }
 else if(data == 'B') {
 Serial.println("\nRandom play");
 random_play();
 }
 // .....
 else if(data == 'D') {
 Serial.println("\nQuerry status of the module");
 query_status();
 }
 }
 }
 delay(100);
}
```

L'esquisse commence par inclure une bibliothèque appelée "*SoftwareSerial.h*".

Nous définissons ensuite cinq macros. Ces macros représentent les octets de commande qui sont les mêmes pour toutes les commandes. Le premier octet est appelé "*Start_Byte*" dont la valeur est *0x7E*, le deuxième octet est appelé "*Version_Byte*" dont la valeur est *0xFF*, le troisième octet est appelé "*Command_Length*" dont la valeur est *0x06*, le quatrième octet est appelé "*End_Byte*" dont la valeur est *0xEF* et le cinquième octet est appelé "*Acknowledge*" dont la valeur *est 0x01*.

Ensuite, nous instancions un objet logiciel sériel appelé "*mySerial*" avec cette ligne de code : `SoftwareSerial mySerial(6, 7) ;`
Où *6* représente la broche numérique I/0 de l'Atmega328p sur laquelle la broche *RX* du module est connectée et *7* représente la broche numérique I/0 de l'Atmega328p sur laquelle la broche *TX* du module est connectée.

Nous créons ensuite un tableau appelé "*receive_buffer*", qui comporte dix éléments. Les éléments du tableau "*receive_buffer*" représentent les octets envoyés par le module et reçus par l'Atmega328p.

Ensuite, nous créons trois variables. La première s'appelle "*data*" et est utilisée pour stocker les commandes lorsque nous les envoyons depuis le moniteur série. La deuxième variable s'appelle "*volume*" et sert à stocker le niveau de volume actuel, lorsque nous envoyons la commande "*Mute*". La troisième variable est appelée "*mute_state*" et est utilisée pour basculer l'état de sourdine du module.

Nous créons ensuite plusieurs fonctions. La première fonction s'appelle "*execute_CMD()*" ; elle accepte trois arguments et ne renvoie aucune valeur. La fonction *execute_CMD()* est utilisée pour envoyer des commandes au module. Le premier argument est l'octet de commande, le deuxième est l'octet de *données 1* et le troisième est l'octet de *données 2* de la commande. Au début de la fonction *execute_CMD()*, nous calculons les octets de somme de contrôle avec cette ligne de code :

```
word checksum = -(Version_Byte + Command_Length + CMD +
                  Accusé de réception + Par1 + Par2) ;
```

Nous créons ensuite un tableau de commandes, appelé "*command_line*". Ce tableau comporte dix éléments, qui représentent dix octets de la commande : *Start_Byte*, *Version_Byte*, *Command_Length*, *Command*, *Acknowledge*, *Data1*, *Data2*, *highByte(Checksum)*, *lowByte(Checksum)*, et *End_Byte*. À la fin de la fonction *execute_CMD()*, nous utilisons une boucle *for pour* envoyer les dix octets, un par un, au module via le logiciel série.

La fonction suivante s'appelle "*reset_rec_buf()*" et est utilisée pour remettre toutes les valeurs des éléments du *tampon de réception* à zéro, ou pour réinitialiser le tampon. La fonction *reset_rec_buf()* n'accepte aucun argument et ne renvoie aucune valeur.

Nous créons ensuite une fonction appelée "*receive()*", qui est utilisée pour recevoir des octets du module et les stocker dans le tableau *receive_buffer*. La fonction *receive()* n'accepte aucun argument et renvoie une valeur booléenne. Au début de la fonction *receive()*, nous appelons *reset_rec_buf()* pour réinitialiser le tableau *receive_buffer*. Ensuite, nous vérifions s'il y a des données sur le

logiciel série, et si ces données ont dix octets.

Si les données de retour comportent dix octets, nous utilisons la boucle *for pour* lire les dix octets. Après avoir lu un octet, nous vérifions si sa valeur est valide en vérifiant si elle est différente de "*-1*". Si c'est le cas, nous stockons sa valeur dans le *tampon de réception*. Si l'une des vérifications n'est pas satisfaite, la valeur booléenne renvoyée est "*false*" ; dans le cas contraire, cette valeur est "*true*". Lorsque nous réinitialisons le module par logiciel, les éléments du *tampon de réception* sont décalés. Nous procédons à cette correction à l'aide du code suivant :

```
short b = receive_buffer[0] ;
for(uint8_t i = 0 ; i < 10 ; i++)
{
  if(i == 9) {
    receive_buffer[i] = b ;
  }
  else {
    receive_buffer[i] = receive_buffer[i+1] ;
  }
}
```

Ensuite, nous créons une fonction appelée "*print_received()*" qui est utilisée pour imprimer les données reçues sur le moniteur série. La fonction *print_received()* accepte un argument, une valeur booléenne qui est utilisée pour déterminer si les données doivent être imprimées ou non. Si la valeur de l'argument est égale à "*true*", nous appelons la *fonction receive()* et imprimons les données du *tampon de réception*. Si la valeur de l'argument est égale à "*false*", nous appelons uniquement la *fonction receive()* sans imprimer les données.

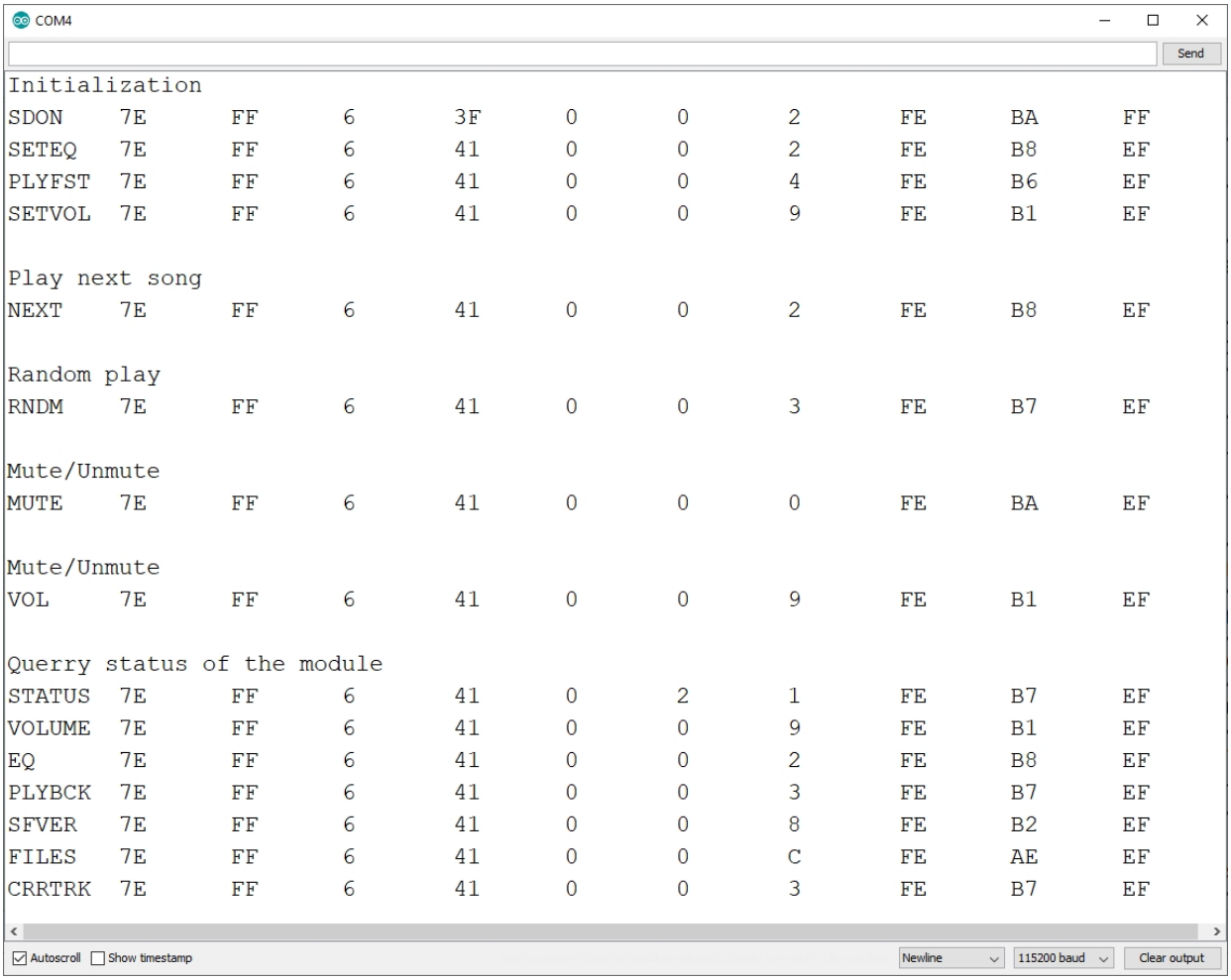
Après ces fonctions, nous créons plusieurs autres fonctions qui utilisent les fonctions précédentes. Toutes ces nouvelles fonctions sont explicites.

Dans la fonction `setup()`, nous démarrons la série matérielle avec un débit en bauds de *115.200* bps, et la série logicielle avec un débit en bauds de *9.600* bps (qui est le débit en bauds par défaut du module). Ensuite, nous appelons la fonction `module_init()` qui initialise le module, règle l'égaliseur, le niveau de volume, joue la première chanson sur le périphérique de stockage et imprime les données d'état sur le moniteur série.

Dans la fonction loop(), nous attendons des données sur le matériel série. Ces données sont envoyées par le moniteur série lorsque nous envoyons une commande. Les données sont des lettres parmi les suivantes : N, B, D et plusieurs autres lettres. Nous vérifions quelle lettre est envoyée et appelons ensuite la fonction correspondante.

Le code de l'esquisse dans cet eBook est juste un exemple, une partie de notre exemple d'esquisse. Si vous voulez voir l'esquisse complète, visitez le dépôt sur le lien *GitHub* suivant : https://github.com/Slaveche90/DFPlayer_Custom_Sketch

Lorsque vous avez téléchargé le sketch complet sur l'Atmega328p, démarrez le Serial Monitor (*Tools* > *Serial Monitor*), et envoyez quelques lettres du sketch via le Serial Monitor à l'Atmega328p. La sortie doit ressembler à celle de l'image ci-dessous :



Exemples de commandes

Commandement	Octets (HEX) *	Description
Chanson suivante	7E FF 06 **01** 00 00 00 EF	Jouer la chanson suivante
Chanson précédente	7E FF 06 **02** 00 00 00 EF	Jouer la chanson précédente
Jouer avec l'index	7E FF 06 **03** 00 **00 01** EF	Jouer la première chanson
	7E FF 06 **03** 00 **00 02** EF	Jouer la deuxième chanson
Augmentation du volume	7E FF 06 **04** 00 00 00 EF	Augmentation du volume d'un niveau
Diminution du volume	7E FF 06 **05** 00 00 00 EF	Diminution du volume d'un niveau
Régler le volume	7E FF 06 **06** 00 **00 1E** EF	Régler le volume sur 30 (=0x1E)
Régler l'égaliseur	7E FF 06 **07** 00 00 **02** EF	Réglez l'égaliseur sur 02 - Rock ; 00 / 01 / 02 / 03 / 04 / 05 Normal/Pop/Rock/Jazz/Classique/Base
Mise en boucle d'une chanson spécifique	7E FF 06 **08** 00 **00 01** EF	Boucle du morceau 0001
Sélectionner un appareil	7E FF 06 **09** 00 00 **01** EF	Sélection d'un périphérique de stockage pour la mémoire USB
	7E FF 06 **09** 00 00 **02** EF	Sélectionner le périphérique de stockage sur la carte SD
Mode veille	7E FF 06 **0A** 00 00 00 EF	La puce passe en mode veille
Réveillez-vous	7E FF 06 **0B** 00 00 00 EF	Chip se réveille
Remise à zéro	7E FF 06 **0C** 00 00 00 EF	Réinitialisation de la puce
Jouer	7E FF 06 **0D** 00 00 00 EF	Reprendre la lecture
Pause	7E FF 06 **0E** 00 00 00 EF	La lecture est interrompue
Jouer une chanson spécifique dans un dossier contenant 256 chansons ; le module contient 256 dossiers. (0 - 255) avec 255 chansons.	7E FF 06 **0F** 00 **01 01** EF	Jouer la chanson avec le dossier : 01/0001xxx.mp3
	7E FF 06 **0F** 00 **01 02** EF	Jouer la chanson dans le dossier : 01/0002xxx.mp3
Amplification audio	7E FF 06 **10** 00 **01 0A** EF	01 - Amp ON ; 0A - niveau (0-31)
	7E FF 06 **10** 00 00 **00** EF	00 - Ampère éteint
Bouclez tout	7E FF 06 **11** 00 00 **01** EF	Démarrer la boucle de toutes les chansons
	7E FF 06 **11** 00 00 **00** EF	Arrêt de la mise en boucle de toutes les chansons et arrêt de la lecture
Lecture dans un dossier mp3	7E FF 06 **12** 00 **00 01** EF	Lecture de la chanson 0001 dans le dossier mp3 (0x0001 - 0x0BB8 ; 3000 chansons)
Jouer une annonce	7E FF 06 **13** 00 **00 01** EF	Lecture de la chanson 0001 dans le dossier ADVERT (0x0001 - 0x0BB8 ; 3000 chansons)

*** Octets de commande sans deux octets de somme de contrôle**

Commandement	Octets (HEX) *	Description
Jouer une chanson spécifique dans un dossier contenant 3000 chansons ; le module supporte 16 dossiers (0 - 15) avec 3000 chansons.	7E FF 06 **14** 00 **00 01** EF	Dans le dossier 0, lecture de la chanson 001
	7E FF 06 **14** 00 **91 11** EF	Dans le dossier 9, lecture du morceau 273 (=0x111)
	7E FF 06 **14** 00 **F0 05** EF	Dans le dossier 15 (=0xF), jouer le morceau 005
Arrêter de jouer Ajouter	7E FF 06 **15** 00 00 00 EF	Arrêter la lecture de l'annonce et reprendre la lecture précédente
Activer la boucle tous	7E FF 06 **16** 00 00 **01** EF	Activer le bouclage de tous les morceaux et commencer la lecture du morceau 1
Arrêter la lecture	7E FF 06 **16** 00 **00 00** EF	Arrêter la lecture
Boucle chanson dans dossier qui prend en charge 256 chansons	7E FF 06 **17** 00 **01 02** EF	Boucle du morceau 02 dans le dossier 01
Lecture aléatoire	7E FF 06 **18** 00 00 00 EF	Lecture aléatoire de toutes les chansons de l'appareil
Régler la lecture d'une seule boucle	7E FF 06 **19** 00 **00 00** EF	Démarrer la lecture en boucle de la chanson en cours
	7E FF 06 **19** 00 00 **01** EF	Arrêter la lecture en boucle de la chanson en cours
Régler le DAC	7E FF 06 **1A** 00 00 **00** EF	Démarrage de la sortie DAC
	7E FF 06 **1A** 00 00 **01** EF	Arrêt de la sortie DAC
Jouer une chanson spécifique avec le volume	7E FF 06 **22** 00 **1E 01** EF	Réglez le volume sur 30 (0x1E est 30) et jouez la première chanson.
	7E FF 06 **22** 00 **0F 02** EF	Réglez le volume à 15 (0x0F correspond à 15) et jouez la deuxième chanson.

*** Octets de commande sans deux octets de somme de contrôle**

Mise à jour de l'état du module

Une option permet d'obtenir les données de retour du module. Ces données sont très utiles car elles peuvent contenir des informations sur l'état de la lecture en cours, le niveau de volume, l'option d'égalisation, la fin du morceau en cours de lecture, etc. Pour activer cette option, vous devez régler l'octet d'*accusé de réception de* la commande sur la valeur *0x01* (*0x00* pas de données de retour).

Lorsque vous envoyez une commande avec l'octet d'*acquittement à 0x01*, les données sont renvoyées. Voici la liste des commandes à envoyer au module pour obtenir des mises à jour d'état :

Octets de commande (HEX) *	Description
7E FF 06 **3F** 00 00 00 EF	Pour obtenir le périphérique de stockage actuel, envoyez la commande suivante
7E FF 06 **40** 00 00 **01** EF	Il s'agit des données de retour, qui indiquent l'erreur, 01 étant la valeur de l'erreur.
7E FF 06 **41** 00 **00 00** EF	Il s'agit de données de retour sans erreur. Ceci indique que la commande a été reçue et exécutée avec succès, où 00 00 est l'état du module.
7E FF 06 **42** 00 00 00 EF	Pour obtenir l'état de la lecture, envoyez la commande suivante
7E FF 06 **43** 00 00 00 EF	Pour obtenir le niveau de volume actuel, envoyez la commande suivante
7E FF 06 **44** 00 00 00 EF	Pour obtenir l'état actuel du QE, envoyez la commande suivante
7E FF 06 **47** 00 00 00 EF	Pour obtenir le nombre total de fichiers sur le disque flash USB, envoyez la commande suivante
7E FF 06 **48** 00 00 00 EF	Pour obtenir le nombre total de fichiers sur la carte SD, envoyez la commande suivante
7E FF 06 **4B** 00 00 00 EF	Pour obtenir le numéro de la chanson en cours sur le disque flash USB, envoyez la commande suivante
7E FF 06 **4C** 00 00 00 EF	Pour obtenir le numéro de la chanson en cours sur la carte SD, envoyez la commande suivante
7E FF 06 **4E** 00 00 00 EF	Pour obtenir le nombre total de fichiers sur n'importe quel support de stockage, envoyez la commande suivante
7E FF 06 **4E** 00 00 **02** EF	Pour obtenir le nombre total de fichiers dans le dossier 02, envoyez la commande suivante
7E FF 06 **4E** 00 00 **0C** EF	Pour obtenir le nombre total de fichiers dans le dossier 12, envoyez la commande suivante

7E FF 06 **4F** 00 00 00 EF	Pour obtenir le nombre total de dossiers sur un périphérique de stockage, envoyez la commande suivante

*** Octets de commande sans deux octets de somme de contrôle**

Valeurs de retour

Les données de retour sont au format :

`0x7E 0xFF 0x06` **`0x41`** `0x00` **`A B`** `checksum1 checksum0 0xEF`

La valeur ***0x41*** indique qu'une ommande a été reçue par le module et exécutée avec succès.

La valeur "***A***" représente le support de stockage,
où : A = *0x01* - disque flash USB, et
A = *0x02* - Carte SD.

La valeur "***B***" indique l'état de la lecture, où B = *0x00* indique que la lecture est arrêtée,
B = *0x01* indique que la lecture est en cours et B = *0x02* indique que la lecture est en pause.

Exemple de données renvoyées :

`0x7E 0xFF 0x06` **`0x41`** `0x00` **`0x02 0x01`** `0xFE 0xF7 0xEF`

où :

`0x02` - le périphérique de stockage est une carte SD

`0x01` - la lecture est en cours

Erreurs

En cas d'erreur, les données renvoyées seront au format :

`0x7E 0xFF 0x06` **`0x40`** `0x00 0x00` **`0x01`** `chks1 chks0 0xEF`

Où *`0x40`* indique qu'une erreur s'est produite, et *`0x01`* indique la valeur de l'erreur. Les valeurs d'erreur et leur description figurent dans le tableau ci-dessous :

Données d'erreur (HEX) *	Description
7E FF 06 **40** 00 00 **01** EF	Le module est occupé
7E FF 06 **40** 00 00 **02** EF	Le module est en mode veille
7E FF 06 **40** 00 00 **03** EF	Erreur de réception en série (la trame n'est pas encore reçue complètement)
7E FF 06 **40** 00 00 **04** EF	Erreur d'incorporation de la somme de contrôle
7E FF 06 **40** 00 00 **05** EF	La chanson spécifiée ne fait pas partie des chansons actuelles
7E FF 06 **40** 00 00 **06** EF	La chanson spécifiée n'est pas trouvée
7E FF 06 **40** 00 00 **07** EF	Erreur d'interférence (la publicité ne peut être jouée que pendant la lecture d'un morceau, et non en pause ou à l'arrêt)
7E FF 06 **40** 00 00 **08** EF	Erreur de lecture de la carte SD (la carte SD est démagnétisée ou retirée)
7E FF 06 **40** 00 00 **0A** EF	Le module est entré en mode veille

*** Octets d'erreur sans deux octets de somme de contrôle**

Données spécifiques renvoyées

Si l'octet d'acquittement est réglé sur `0x01`, le module émet des données lorsque la chanson est terminée, lorsque la carte SD (ou le disque flash USB) est *insérée* ou *retirée* ou lorsque le dispositif de stockage est en ligne. Ces valeurs sont renvoyées sans qu'aucune commande ne soit envoyée au module.

Les données renvoyées par le dispositif de stockage lorsqu'il est poussé vers l'*intérieur* :

`0x7E 0xFF 0x06` **`0x3A`** `0x00 0x00` **`A`** `0xFE 0xF7 0xEF`

où :

`0x3A` indique que l'unité de stockage est poussée vers l'intérieur (*IN)*

Les données renvoyées par le dispositif de stockage lorsqu'il est tiré vers l'*extérieur* :

`0x7E 0xFF 0x06` **`0x3B`** `0x00 0x00` **`A`** `0xFE 0xF7 0xEF`

où :

`0x3B` indique que l'unité de stockage est tirée vers l'*extérieur.*

A = `0x01` indique que le dispositif de stockage est un

disque flash USB A = `0x02` indique que le dispositif de

stockage est une carte SD

A = `0x04` indique que le câble USB est connecté ou non au PC.

Les données renvoyées de la chanson finie sont au format suivant :

`0x7E 0xFF 0x06` **`0x3D`** `0x00` **`0x00 0x05`** `0xFE 0xF7 0xEF`

où :

`0x3D` indique que la chanson est terminée sur la carte SD (`0x3C` = sur le

disque flash USB),

`0x00 0x05` indique le nom du morceau "`0005`".

Les données renvoyées lorsque le dispositif de stockage est en ligne :

`0x7E 0xFF 0x06` **`0x3F`** `0x00 0x00` **`A`** `0xFE 0xF7 0xEF`

où :

`0x3F` indique que l'unité de stockage est en ligne, et

"**`A`**" peut avoir plusieurs valeurs différentes :

A = *`0x01`* indique un disque flash

USB A = *`0x02`* indique une carte

SD

A = *`0x03`* indique que le disque flash USB et la carte SD sont en ligne en même temps.

A = *`0x04`* indique une connexion PC

Lecture des valeurs retournées

Si l'octet d'accusé de réception est réglé sur *0x01*, nous envoyons la commande d'état de lecture :

`0x7E 0xFF 0x06` **`0x45`** `0x00 0x00 0x00 chks1 chks0 0xEF`

Les données renvoyées seront au format :

`0x7E 0xFF 0x06` **`0x41`** `0x00 0x00` **`A`** `chks1 chks0 0xEF`

où "***A***" peut avoir plusieurs valeurs différentes :

A = *0x00* indique que la lecture est réglée pour lire et mettre en boucle toutes les chansons du périphérique de stockage, une par une,

A = *0x01* indique que la lecture est réglée pour lire et mettre en boucle tous les morceaux d'un dossier spécifique, un par un,

A = *0x02* indique que la lecture est réglée sur la lecture et la mise en boucle d'un morceau,

A = *0x03* indique que la lecture est réglée sur la lecture aléatoire et met en boucle toutes les chansons sur le périphérique de stockage ; en lecture aléatoire, les chansons sont répétées,

A = *0x04* indique que la lecture est réglée pour jouer un morceau et que lorsque le morceau est terminé, la lecture s'arrête.

Vous avez réussi !

Vous pouvez maintenant utiliser votre module pour divers projets.

Le moment est venu d'apprendre et de réaliser les projets par vous-même. Vous pouvez le faire avec l'aide de nombreux scripts d'exemple et d'autres tutoriels, que vous pouvez trouver sur Internet.

Si vous êtes à la recherche de produits microélectroniques et d'accessoires de haute qualité, AZ-Delivery Vertriebs GmbH est l'entreprise qu'il vous faut. Vous disposerez de nombreux exemples d'application, de guides d'installation complets, de livres électroniques, de bibliothèques et de l'assistance de nos experts techniques.

https://az-delivery.de

Amusez-vous !

Impressum

https://az-delivery.de/pages/about-us